Computer Vision:

Homework-8

Bilal Ahmed
ahmedb@purdue.edu

1 Theoretical Questions

theoretical observation fundamental to Zhang's algorithm

Answer: Since we cannot directly image the absolute conic, the observation that π samples the absolute conic at exactly the circular points is really useful for solving for ω. This is like trying to figure out ω by sampling two points for each plane π. Using these circular points, we can solve for ω and then using $\omega = K^{-T}K^{-1}$ we can solve for intrinsic parameters of camera.

Image of the Absolute Conic Ω_∞

Answer: From camera modeling equation we have,

$$x = K[R|t]X$$

For point X at infinity,

$$x = K[R|t]\begin{bmatrix} u \\ v \\ w \\ 0 \end{bmatrix} = K[R|t]\begin{bmatrix} x_d \\ 0 \end{bmatrix}$$

where $x_d = \begin{bmatrix} u \\ v \\ w \end{bmatrix}$ is the direction vector towards the points at infinity. This simpifies as;

$$x = KRx_d = Hx_d$$

where $H = KR$

For point x_d on absolute conic, it satisfies;

$$x_d^T H x_d = 0$$

We also know that a conic C is transformed by homography H as $C' = H^{-T}CH^{-1}$ There for transformation (image) of absolute conic ω is given be;

$$\omega = H^{-T}IH^{-1} = (KR)^{-T}I(KR)^{-1}$$

which simplifies to;

$$\omega = K^{-T}K^{-1}$$

The points lying on this conic ω must satisfy $x^T\omega\, x = 0$ where we know that K is upper triangular and $K^{-T}K^{-1}$ is positive definite implying that $x^T\omega\, x > 0$. That means the points lying on ω must be imaginary.

2 Zhang's Algorithm

Zhang's Algorithm is used to calibrate camera i.e. calculating the intrinsic and extrinsic parameters of the camera. For that purpose we use the calibration pattern; for example checkerboard pattern provided for this assignment. Multiple images of the calibration pattern are taken from different views. These images are used by this algorithm to do calibration. The steps are detailed here.

2.1 Corner Detection

First step is to detect the corners in each view. Here are the steps of my implementation of corners detection.

- I use the Otsu's algorithm for segmentation of the checkerboard pattern in the image from the background.
- Edge detection is sensitive to noise; to get rid of noise coming from the surroundings of the patter, I used morphological operators erosion followed by dilation. The resulting mask has 255's (high) at the checkerboard and 0's outside it. So I invert the mask to get 0's in the checkerboard.
- Then edges are extracted using Canny edge detector with lower threshold of 100, upper threshold of 400 and aperture size of 7. I used cv2.Canny function from OpenCV.
- Hough's method is applied for detecting the horizontal and vertical lines in the mask. I used cv2.HoughLines method with voting threshold of 60.
- At this point we have many horizontal and vertical lines. I segregate them into horizontal and vertical based on the θ. Specifically, lines having $\frac{\pi}{4} < \theta < \frac{3\pi}{4}$ are labeled as horizontal and rest of them as vertical.
- Vertical and horizontal lines are then clustered into 8 and 10 clusterd using density based spatial clustering (DBSCAN) based on Euclidean distance. I choose the clustering threshold adaptively until the desired number of clusters are found. Specifically, threshold is halved if number of clusters are less than desired and multiplied by 1.5 if the clusters are greater than the desired. I found that clustering was more robust when done based on cartesian coordinates i.e. $x = \rho\cos(\theta)$ and $y = \rho\sin(\theta)$
- Next the horizontal and vertical lines are sorted based on $y = \rho\sin(\theta)$ and $x = \rho\cos(\theta)$ components respectively.
- Intersection points of horizontal and vertical lines give us the corner points numbered in the raster order.

2.2 Estimating the intrinsic parameters

Intrinsic parameters are estimated using the image of absolute conic. Since imaging the absolute conic is not possible; because it is just an abstract concept, We use different views to take samples of absolute conic. From camera modeling, we have the relationship between homogeneous representation of point in world coordinates and corresponding homogeneous representation of pixel coordinates.

$$x = K[R|t]X$$

where x is the homogeneous coordinates of pixel coordinates and X if homogeneous representation of point in world. We pick our world coordinate reference at the top left corner of the calibration pattern, so the pattern is in Z=0 plane. Then for corners of the calibration pattern, we can write;

$$x = K[R|t] \begin{bmatrix} a \\ b \\ 0 \\ c \end{bmatrix}$$

We can write this expression using the homography matrix H as;

$$x = Hx_w$$

Camera image of absolute conic ω is given as;

$$\omega = K^{-T}K^{-1}$$

For the points on the absolute conic, we can write the conic expression $x^T\omega\, x$ as

$$\mathbf{h}_1^T\, \omega \mathbf{h}_1 = \mathbf{h}_2^T\, \omega \mathbf{h}_2$$

$$\mathbf{h}_1^T\, \omega \mathbf{h}_2 = 0$$

Since ω is symmetric matrix, we can write the above two equations as matrix vector product;

$$\mathbf{v}_{12}^T \mathbf{b} = 0$$

$$(\mathbf{v}_{11} - \mathbf{v}_{22})^T \mathbf{b} = 0$$

where $\mathbf{v}_{ij} = \begin{bmatrix} h_{i1}h_{j1} \\ h_{i1}h_{j2} + h_{i2}h_{j1} \\ h_{i2}h_{j2} \\ h_{i3}h_{j1} + h_{i1}h_{j3} \\ h_{i3}h_{j2} + h_{i2}h_{j3} \\ h_{i3}h_{j3} \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} \omega_{11} \\ \omega_{12} \\ \omega_{22} \\ \omega_{13} \\ \omega_{23} \\ \omega_{33} \end{bmatrix}$ Using all the correspondences of corner points, we can write; Vb = 0 where V matrix is concatenation of all matrices of coefficient for all correspondences. This can be solved using linear least-squares. Solving this we get ω then using $\omega = K^{-T}K{-}1$ we can get matrix of intrinsic parameters K. Where

$$K = \begin{bmatrix} \alpha_x & s & x_0 \\ 0 & \alpha_y & y_0 \\ 0 & 0 & 1 \end{bmatrix}$$

Zhang's algorithm gives us solution to the parameters in K by introducing another variable λ to account of homogeneous object on one side of equation and non-homogeneous objects on the other side.

$$y_0 = \frac{\omega_{12}\omega_{13} - \omega_{11}\omega_{23}}{\omega_{11}\omega_{22} - \omega_{12}^2}$$

$$\lambda = \omega_{33} - \frac{\omega_{13}^2 + y_0(\omega_{12}\omega_{13} - \omega_{11}\omega_{23})}{\omega_{11}}$$

$$\alpha_x = \sqrt{\frac{\lambda}{\omega_{11}}}$$

$$\alpha_y = \sqrt{\frac{\lambda\, \omega_{11}}{\omega_{11}\omega_{22} - \omega_{12}^2}}$$

$$s = -\frac{\omega_{12}\, \alpha_x^2 \alpha_y}{\lambda}$$

$$x_0 = \frac{s y_0}{\alpha_y} - \frac{\omega_{13}\, \alpha_x^2}{\lambda}$$

2.3 Estimating the extrinsic parameters

As we saw earlier that camera modeling equation can be written in terms of homography H, specifically we can write;

$$H = K[R|t]$$

$$\begin{bmatrix} h1 & h2 & h3 \end{bmatrix} = K \begin{bmatrix} r1 & r2 & t \end{bmatrix}$$

From there we can see that,

$$r1 = \frac{K^{-1}h1}{\|K^{-1}h1\|}$$

$$r2 = \frac{K^{-1}h2}{\|K^{-1}h1\|}$$

$$r3 = r1 \times r2$$

$$t = \frac{K^{-1}h3}{\|K^{-1}h1\|}$$

This gives a rotation matrix R and translation vector t. To make sure the columns of rotation matrix are orthogonal, we condition the matrix using singular value decomposition, specifically, we set sigular values 1.

2.4 Refining parameters using Levenberg–Marquardt

Normally we have many view available to calibrate the camera, we can use them all to quantify the goodness of our estimate. Specifically, we can project coordinates of physical points to every view and compute geometric error using;

$$d^2_{\text{geom}} = \|X - f(p)\|^2_2$$

where f(p) are the projected points.

3 Results

Two datasets were used for this assignment. Dataset-1 was provided by the instructor and dataset-2 was created by myself following the instruction given in the handout.

3.1 Dataset-1

There are total 40 views in dataset-1. Fig.1, Fig.2 and Fig.3 show the intermediate steps of corner detection algorithm. Fig.4 shows projections of physical coordinates onto three different views using the linear estimates as well as estimates refined using LM. Fig.5 shows visualization of camera poses in front of calibration pattern. Here are the parameters of camera estimates using dataset-1;

$$K = \begin{bmatrix} 719.56826589 & 1.82552776 & 325.92337184 \\ 0 & 718.50025888 & 242.3413243 \\ 0 & 0 & 1 \end{bmatrix}$$

$$K_{(\text{LM})} = \begin{bmatrix} 720.56434973 & 2.06573398 & 326.87150931 \\ 0 & 719.69462391 & 243.99150623 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R_5 = \begin{bmatrix} 0.9484325 & -0.14050279 & -0.28413863 \\ 0.11321228 & 0.98742127 & -0.11037303 \\ 0.29607225 & 0.07251339 & 0.95240907 \end{bmatrix}$$

$$t_5 = \begin{bmatrix} -8.24771148 \\ -14.86711545 \\ 48.61742188 \end{bmatrix}$$

$$R_5(LM) = \begin{bmatrix} 0.94824801 & -0.13975214 & -0.28512288 \\ 0.11329285 & 0.9877446 & -0.10735615 \\ 0.29663183 & 0.06949787 & 0.95245976 \end{bmatrix}$$

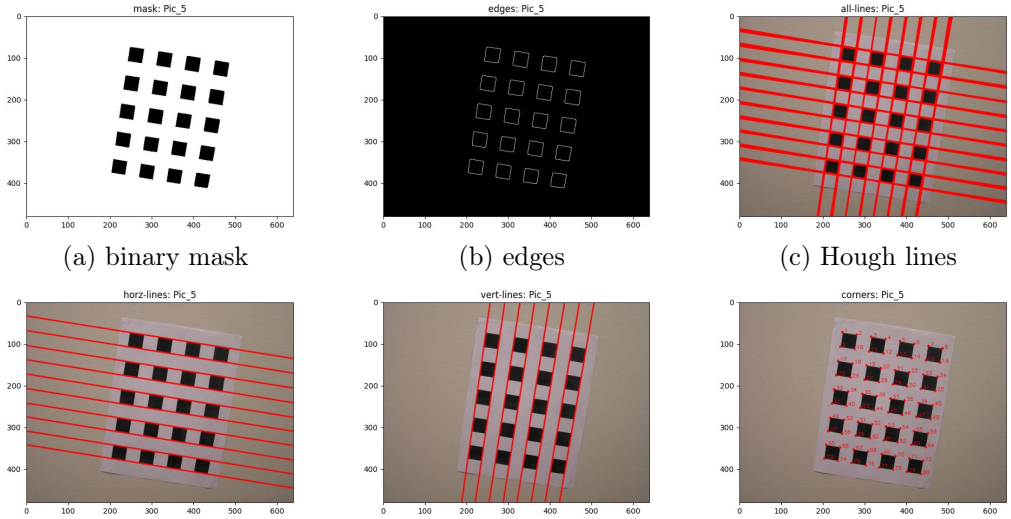
$$t_5(LM) = \begin{bmatrix} -8.30852641 \\ -14.97494038 \\ 48.74103479 \end{bmatrix}$$

$$R_{14} = \begin{bmatrix} 0.96941386 & -0.00389874 & 0.24540084 \\ 0.01341597 & 0.99922066 & -0.03712265 \\ -0.24506485 & 0.03927951 & 0.96871066 \end{bmatrix}$$

$$t_{14} = \begin{bmatrix} -9.8437625 \\ -12.19532275 \\ 60.16793803 \end{bmatrix}$$

$$R_{14}(LM) = \begin{bmatrix} 0.96854351 & -0.00454877 & 0.24880268 \\ 0.01373589 & 0.99928583 & -0.03520173 \\ -0.24846487 & 0.03751193 & 0.96791428 \end{bmatrix}$$

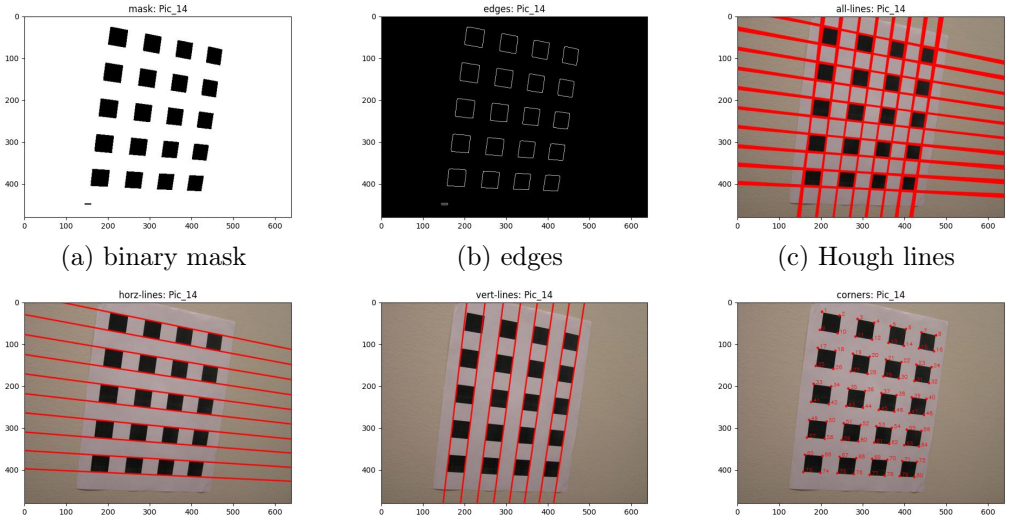
$$t_{14}(LM) = \begin{bmatrix} -9.9109336 \\ -12.30836938 \\ 60.21039336 \end{bmatrix}$$



(a) binary mask (b) edges (c) Hough lines

(d) horizontal lines (e) vertical lines (f) corners

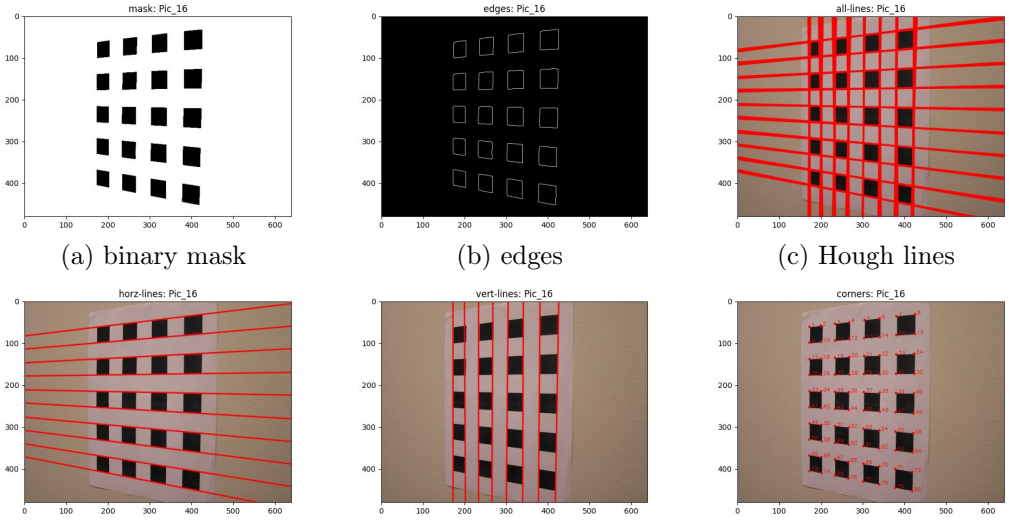
Figure 1: Corners detection steps for dataset1, view-5 (randomly selected).



(a) binary mask (b) edges (c) Hough lines

(d) horizontal lines (e) vertical lines (f) corners

Figure 2: Corners detection steps for dataset1, view-14 (randomly selected).



(a) binary mask (b) edges (c) Hough lines

(d) horizontal lines (e) vertical lines (f) corners

Figure 3: Corners detection steps for dataset1, view-16 (randomly selected).

3.2 Dataset-2

There are total 20 views in dataset-2. Fig.6, Fig.7 and Fig.8 show the intermediate steps of corner detection algorithm. Fig.9 shows projections of physical coordinates onto three different views using the linear estimates as well as estimates refined using LM. Fig.10 shows visualization of camera poses in front of calibration pattern.

Table.1 shows summary of geometric errors for different views from dataset-1 and dataset-2.

$$K = \begin{bmatrix} 5.44158389 \times 10^2 & -5.31714665 \times 10^{-2} & 2.89725489 \times 10^2 \\ 0 & 5.43360935 \times 10^2 & 3.89634847 \times 10^2 \\ 0 & 0 & 1 \end{bmatrix}$$

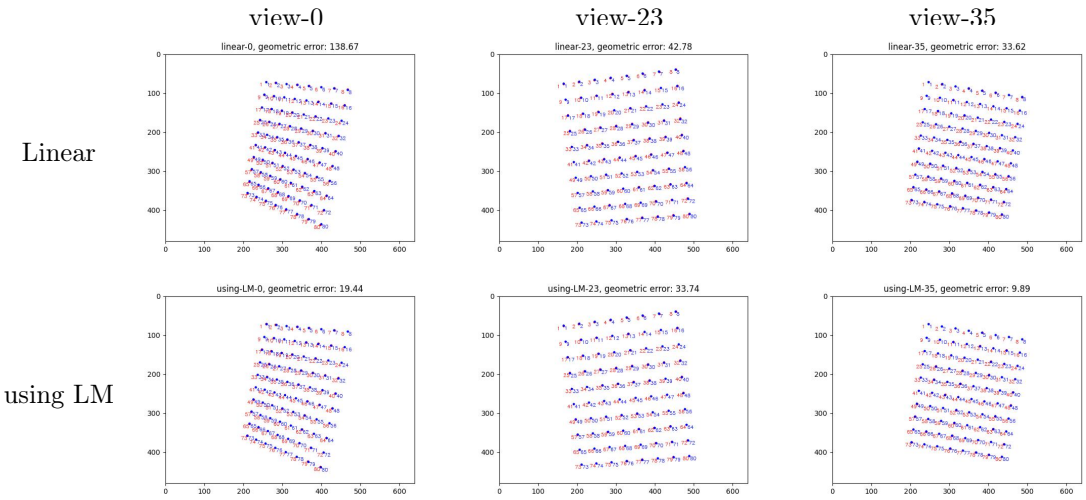


Figure 4: Corners detected (red) and physical corners projected (blue) to different views (randomly selected). Geometric Error between true and projected corners is shown in the title.

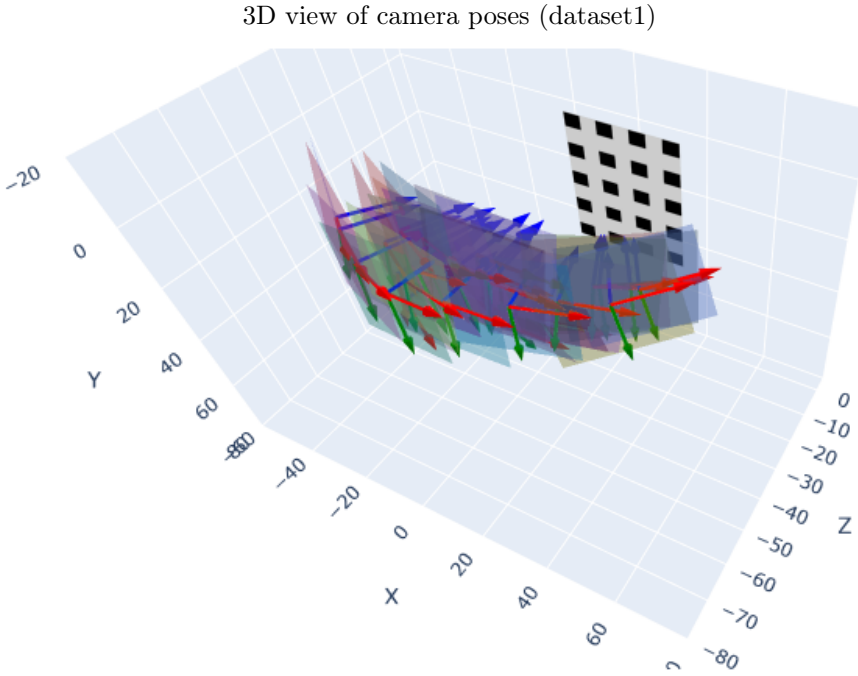


Figure 5: Visualization of camera poses for all the pictures in dataset-1 in front of calibration pattern. Principle plane for each camera pose is displayed as 2d planar surface and three camera axis are displayed at red, green and blue axis.

$$K_{(\mathrm{LM})} = \begin{bmatrix} 5.48344409 \times 10^2 & 6.95585950 \times 10^{-2} & 2.90087556 \times 10^2 \\ 0 & 5.47524898 \times 10^2 & 3.90001650 \times 10^2 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R_3 = \begin{bmatrix} 0.90581263 & 0.00385503 & 0.42366097 \\ 0.0721846 & 0.98393413 & -0.16328815 \\ -0.41748397 & 0.17849026 & 0.89098168 \end{bmatrix}$$

$$t_3 = \begin{bmatrix} -11.00804304 \\ -11.25223112 \\ 36.58376119 \end{bmatrix}$$

$$R_3(LM) = \begin{bmatrix} 0.90532961 & 0.00380899 & 0.42469259 \\ 0.07287773 & 0.98373484 & -0.16417854 \\ -0.41841025 & 0.17958632 & 0.89032669 \end{bmatrix}$$

$$t_3(LM) = \begin{bmatrix} -11.01704557 \\ -11.27507665 \\ 36.82639644 \end{bmatrix}$$

$$R_6 = \begin{bmatrix} 0.95635611 & -0.0130151 & 0.29191368 \\ 0.02798041 & 0.99849586 & -0.04714996 \\ -0.29086094 & 0.05326001 & 0.95528178 \end{bmatrix}$$

$$t_6 = \begin{bmatrix} -12.00576368 \\ -14.49751808 \\ 42.25603547 \end{bmatrix}$$

$$R_6(LM) = \begin{bmatrix} 0.95614835 & -0.01294475 & 0.29259661 \\ 0.02836931 & 0.99841855 & -0.04853434 \\ -0.29150562 & 0.05470679 & 0.95500348 \end{bmatrix}$$

$$t_6(LM) = \begin{bmatrix} -12.02119716 \\ -14.52310271 \\ 42.54986852 \end{bmatrix}$$

For dataset-2, view-0 is directly in front of the calibration pattern, roughly at a distance of around 1 foot from the calibration pattern. If we take a loot at its extrinsic parameters;

$$R_0(LM) = \begin{bmatrix} 0.99856635 & 0.00275927 & 0.05345689 \\ 0.00407248 & 0.99185977 & -0.12726982 \\ -0.05337291 & 0.12730506 & 0.99042655 \end{bmatrix}$$

$$t_0(LM) = \begin{bmatrix} -10.31974375 \\ -12.41020929 \\ 33.54369011 \end{bmatrix}$$

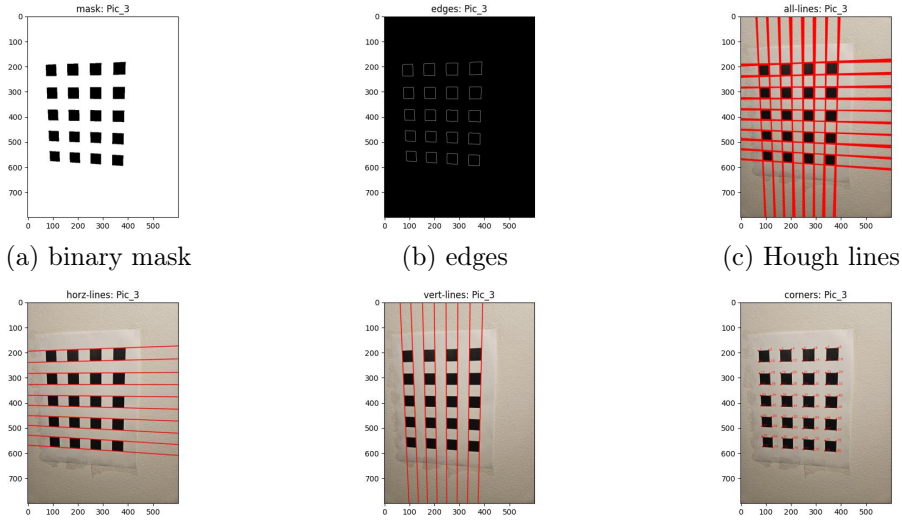
Using these we can compute coordinates of camera center using $C = -R^T t$

$$\text{Camera center} = \begin{bmatrix} 12.14581356 \\ 8.06738083 \\ -34.25034498 \end{bmatrix}$$

The coordinates of camera center show distance from calibration plan at z=-34.25 cm, which is we expected. This proves the accuracy of the computed parameters.

Python Code

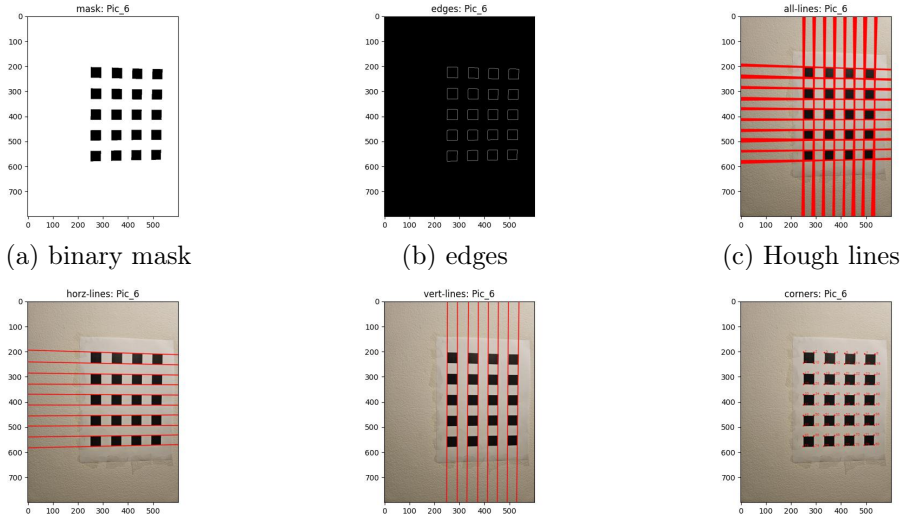
```
import cv2
import pathlib
# import skimage
import sklearn.cluster
import scipy
import computer_vision
import BitVector
```



(a) binary mask (b) edges (c) Hough lines

(d) horizontal lines (e) vertical lines (f) corners

Figure 6: Corners detection steps for dataset2, view-3 (randomly selected).

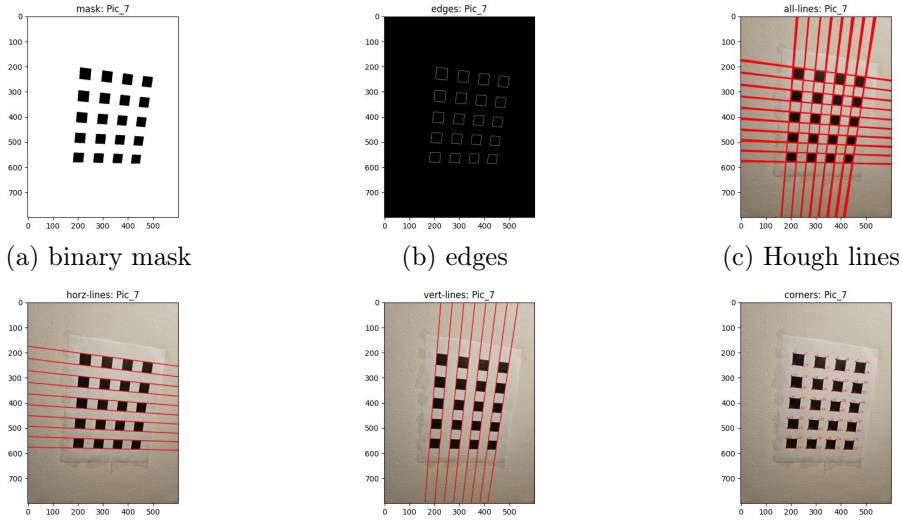


(a) binary mask (b) edges (c) Hough lines

(d) horizontal lines (e) vertical lines (f) corners

Figure 7: Corners detection steps for dataset2, view-6 (randomly selected).

```
8
9  import matplotlib.pylab as plt
10 import matplotlib as mpl
11 import plotly.graph_objects as go
12 import numpy as np
13 import math
14
15 import os
16 import glob
17 import time
18 import torch
19 from models.matching import Matching
20 from operator import itemgetter
```



(a) binary mask (b) edges (c) Hough lines

(d) horizontal lines (e) vertical lines (f) corners

Figure 8: Corners detection steps for dataset2, view-7 (randomly selected).

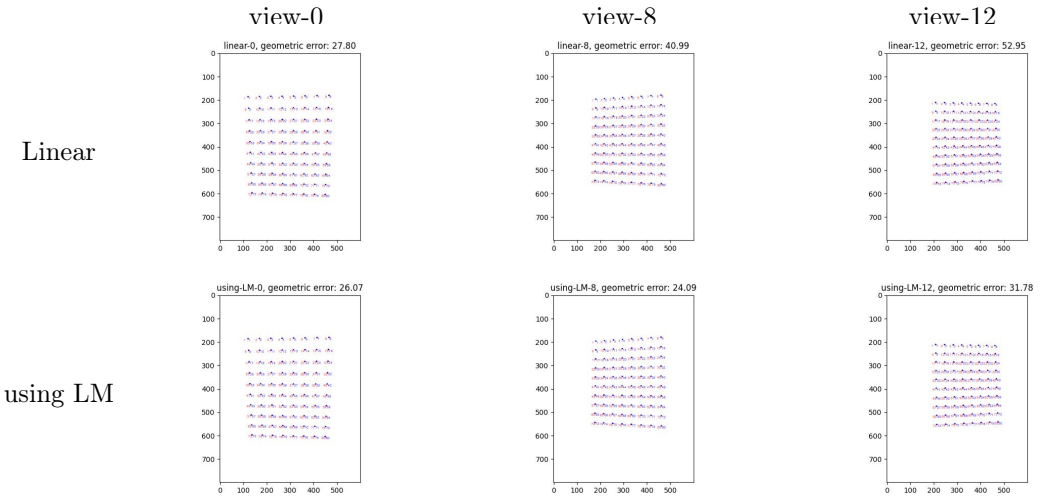


Figure 9: Corners detected (red) and physical corners projected (blue) to different views (randomly selected). Geometric Error between true and projected corners is shown in the title.

Table 1: Geometric error between actual corner points and projected corner points in three views for each dataset.

dataset	view	error (w/o LM)	error (LM)
dataset-1	view-0	138.67	19.44
	view-23	42.78	33.74
	view-35	33.62	9.89
dataset-2	view-0	27.8	26.07
	view-8	40.99	24.09
	view-12	52.95	31.78

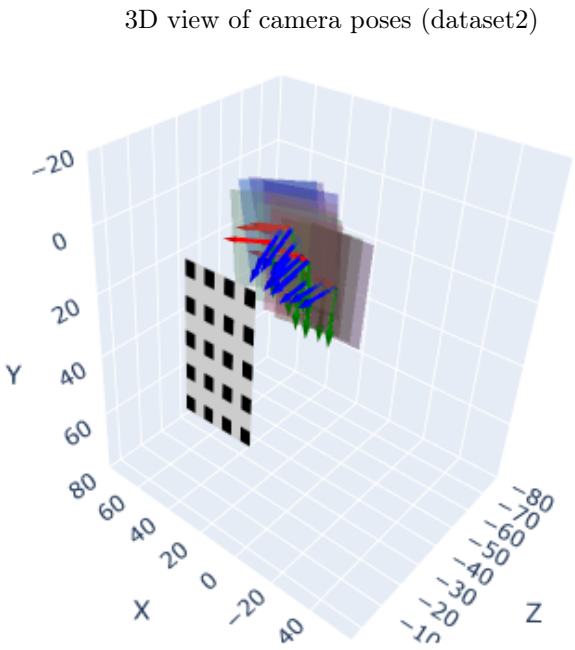


Figure 10: Visualization of camera poses for all the pictures in dataset-2 in front of calibration pattern. Principle plane for each camera pose is displayed as 2d planar surface and three camera axis are displayed at red, green and blue axis.

```
from models.utils import (AverageTimer, VideoStreamer,
                          make_matching_plot_fast, frame2tensor)


from sklearn.svm import SVC, LinearSVC
from sklearn.metrics import accuracy_score, confusion_matrix
import seaborn as sns

torch.set_grad_enabled(False)
import matplotlib

def read_image_from_path(image_path):
        """Reads images from the subdir"""
        img_bgr = cv2.imread(image_path)
        # Convert the image from BGR to RGB format
        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
        return img_rgb


def otsu_single_itr(img, L=256, mask=None):
    """Performs one iteration of otsu algorithm.

    Args:
        img: single level image
        L: number of of gray levels
        mask: mask to focus on image regions

    Returns:
```

```
        threshold
        gray_level_index
    """
    epsilon = 1.e-6
    if mask is None:
        mask = np.where(img > 0)
    bins = np.linspace(0, 255, num=L)
    prob_dist, bins = np.histogram(img[mask].flatten(), bins=bins, density=True)
    # getting upper bin edges
    gray_levels = bins[1:]
    mu_t = np.sum(gray_levels*prob_dist)
    max_var = 0
    threshold = 0
    for k in range(1, L-1):
        mu_k = np.sum(gray_levels[:k]*prob_dist[:k])
        w_k = np.sum(prob_dist[:k])
        var_b = ((mu_t*w_k - mu_k)**2)/(w_k*(1-w_k)+epsilon)        # OTSU threshold selection method: Eq. 18

        if var_b > max_var:
            max_var = var_b
            threshold = gray_levels[k]

    # print(f"threshold: {threshold}, max var_b: {max_var}")
    return threshold


def otsu_single_channel(img, L = 256, max_itr=10, flip=False):
    """Performs multiple iterations of otsu algorithm, stops if number of iterations is equal to the max_itr or probability of foreground
    object is less than or equal to the given estimate of probability.

    Args:
        img: input image (single channel)
        L: number of gray levels, default=256.
        max_itr: max number of iterations, default=5
        flip: If yes, smaller pixels values are considered as foreground.
    """
    mask = None
    for itr in range(max_itr):

        threshold = otsu_single_itr(img, L=L, mask=mask)
        if flip:
            mask = np.where(img < threshold)
        else:
            mask = np.where(img > threshold)

    print(f"Selected threshold: {threshold}")
    segmented_img = np.zeros_like(img)
    segmented_img[mask] = 255
    return segmented_img.astype(np.uint8)
```

```python
def erosion(img, kernel_size=3):
    """Erosion implemented for single channel images,
    convert image to grayscale if it is 3 channel
    """
    if len(img.shape) ==3:
        img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    height, width = img.shape
    k = kernel_size//2
    eroded_img = np.zeros_like(img)
    for i in range(k, height - k):
        for j in range(k, width - k):
            eroded_img[i,j] = np.min(img[i-k:i+k+1, j-k:j+k+1])
    return eroded_img

def dilation(img, kernel_size=3):
    """Dilation implemented for single channel images,
    convert image to grayscale if it is 3 channel
    """
    if len(img.shape) ==3:
        img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    height, width = img.shape
    k = kernel_size//2
    dilated_img = np.zeros_like(img)
    for i in range(k, height - k):
        for j in range(k, width - k):
            dilated_img[i,j] = np.max(img[i-k:i+k+1, j-k:j+k+1])

    return dilated_img


class ClusterManager:
    def __init__(self, num_horz_lines=10, num_vert_lines = 8) -> None:
        self.num_horz_lines = num_horz_lines
        self.num_vert_lines = num_vert_lines

    def euclidean_distance(self, line1, line2):
        """Calculates difference between theta values"""
        return np.sqrt(np.sum((line1 - line2)**2))


    def get_clustered_lines(self, lines):
        """Given all the line parameters, applies clustering separately
        for horizontal and vertical lines to return the parameters of useful
            lines."""
        horz_lines_mask = (lines[:,1] > (np.pi/4)) & (lines[:,1] < (3*np.pi
            /4))

        horz_lines = lines[horz_lines_mask]
        vert_lines = lines[~horz_lines_mask]

        horz_lines = self.get_cartesian_coordinates(horz_lines)
        vert_lines = self.get_cartesian_coordinates(vert_lines)

        horz_useful_lines = self.cluster_adaptively(
            horz_lines, self.num_horz_lines, self.euclidean_distance
            )
        vert_useful_lines = self.cluster_adaptively(
            vert_lines, self.num_vert_lines, self.euclidean_distance
            )
```

```
        horz_useful_lines = self.get_polar_coordinates(horz_useful_lines)
        vert_useful_lines = self.get_polar_coordinates(vert_useful_lines)

        return horz_useful_lines, vert_useful_lines


    def cluster_adaptively(self, lines, n_clusters, distance_metric, num_itr=100, initial_eps=50):
        """Adaptively picks the threshold to cluster the data points into n_clusters."""
        num = 1
        eps = initial_eps
        if distance_metric is None:
            distance_metric = self.rho_distance
        while num_itr>0 and num != n_clusters:
            num_itr -= 1
            clustered_lines = self.density_based_clustering(lines, eps=eps, distance_metric=distance_metric)

            num = clustered_lines.shape[0]
            if num < n_clusters:
                eps = eps*0.5
            elif num > n_clusters:
                eps = eps*1.5
        print(f"eps: {eps}")
        return clustered_lines

    def density_based_clustering(self, lines, eps=50, distance_metric=None):
        """Density based clustering using the threshold eps"""
        if distance_metric is None:
            distance_metric = self.rho_distance

        cluster_obj = sklearn.cluster.DBSCAN(eps=eps, min_samples=1, metric=distance_metric).fit(lines)
        clustered_lines = []
        for label in np.unique(cluster_obj.labels_):
            if label != -1:
                cluster_lines = lines[np.where(cluster_obj.labels_==label)[0]]
                clustered_lines.append(np.median(cluster_lines, axis=0))

        return np.array(clustered_lines)


    def get_cartesian_coordinates(self, lines):
        """Given polar coordinates, returns cartesian coordinates"""
        rhos = lines[:,0]
        angles = lines[:,1]
        x = rhos*np.cos(angles)
        y = rhos*np.sin(angles)
        return np.stack([x,y], axis=1)

    def get_polar_coordinates(self, lines):
        """Given cartesian coordinates, returns polar coordinates"""
        x_coord = lines[:,0]
        y_coord = lines[:,1]

        angles = np.arctan2(y_coord, x_coord)
```

```
        rhos = np.sqrt(x_coord**2 + y_coord**2)

        mask = (angles < 0)
        angles[mask] = angles[mask] + np.pi
        rhos[mask] *= -1

        return np.stack([rhos, angles], axis=1)


class CornersDetector:
    def __init__(self, voting_th=40, canny_th1=200, canny_th2=500,
            min_rho_diff = 10, min_theta_diff=2, output_dir=None) -> None:
        # self.data_dir = data_dir
        self.canny_th1 = canny_th1
        self.canny_th2 = canny_th2
        self.voting_th = voting_th

        self.min_rho_diff = min_rho_diff
        self.min_theta_diff = min_theta_diff

        self.cluster_manager = ClusterManager()
        self.plotter = None
        if output_dir is not None:
            self.plotter = Plotter(output_dir)

    def get_corner_pts(self, image):
        """Returns coordinates of corners points of the calibation pattern.
            """
        pts_horz_lines, pts_vert_lines = self.get_Hough_lines(image)
        pts = CornersDetector.get_intersection_pts(pts_horz_lines,
            pts_vert_lines)
        return pts

    def visualize_Hough_lines_extraction(self, image, image_name, subdir=
        None):
        """ """
        assert self.plotter is not None, "plotter object not availble, make
            sure initialize contructor with output dir"
        img1_gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

        mask = otsu_single_channel(
            img1_gray, L = 256, max_itr=1, flip=True
            )
        mask = erosion(mask)
        mask = dilation(mask)
        mask = 255 - mask

        self.plotter.save_visualization(mask, image_name=image_name,
            identifier='mask', sub_dir=subdir)
        edges = cv2.Canny(mask, self.canny_th1, self.canny_th2, apertureSize
            =7, L2gradient=True)

        self.plotter.save_visualization(edges, image_name=image_name,
            identifier='edges', sub_dir=subdir)
        lines = cv2.HoughLines(edges, 1, 0.1*np.pi / 180, self.voting_th,
            None, 0, 0).squeeze()

        pts_on_lines = CornersDetector.get_pts_on_lines(lines)
        self.plotter.display_img_with_lines(
```

```
            image, pts=pts_on_lines , image_name=image_name, identifier='all-lines',
            sub_dir=subdir
            )

        horz_lines, vert_lines = self.cluster_manager.get_clustered_lines(lines)

        horz_lines = np.array(sorted(horz_lines, key=lambda line: line[0]*np.sin(line[1])))
        vert_lines = np.array(sorted(vert_lines, key=lambda line: line[0]*np.cos(line[1])))
        pts_horz_lines = CornersDetector.get_pts_on_lines(horz_lines)
        pts_vert_lines = CornersDetector.get_pts_on_lines(vert_lines)
        self.plotter.display_img_with_lines(
            image, pts=pts_horz_lines, image_name=image_name, identifier='horz-lines',
            sub_dir=subdir
            )
        self.plotter.display_img_with_lines(
            image, pts=pts_vert_lines, image_name=image_name, identifier='vert-lines',
            sub_dir=subdir
            )

        pts = CornersDetector.get_intersection_pts(pts_horz_lines, pts_vert_lines)
        self.plotter.display_img_with_pts(
            image, pts=pts, image_name=image_name, identifier='corners',
            sub_dir=subdir, label_pts=True
            )


    def get_Hough_lines(self, image):
        """ """
        img1_gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

        mask = otsu_single_channel(
            img1_gray, L = 256, max_itr=1, flip=True
            )
        mask = erosion(mask)
        mask = dilation(mask)

        mask = 255 - mask
        edges = cv2.Canny(mask, self.canny_th1, self.canny_th2, apertureSize=7, L2gradient=True)

        lines = cv2.HoughLines(edges, 1, 0.1*np.pi / 180, self.voting_th, None, 0, 0).squeeze()

        # return lines

        print(f"Number of lines detected: {lines.shape[0]}")

        horz_lines, vert_lines = self.cluster_manager.get_clustered_lines(lines)

        horz_lines = np.array(sorted(horz_lines, key=lambda line: line[0]*np.sin(line[1])))
```

```
315         vert_lines = np.array(sorted(vert_lines, key=lambda line: line[0]*np
                .cos(line[1])))
316         pts_horz_lines = CornersDetector.get_pts_on_lines(horz_lines)
317         pts_vert_lines = CornersDetector.get_pts_on_lines(vert_lines)
318
319         return pts_horz_lines, pts_vert_lines
320
321
322
323     def filter_duplicate_lines(self, lines):
324         filtered_lines = []
325         min_rho_diff = 10
326         min_theta_diff = 2*np.pi / 180
327
328         for i in range(len(lines)):
329             rho, theta = lines[i]
330             duplicate = False
331             for rho2, theta2 in filtered_lines:
332                 circular_theta_diff = min(abs(theta - theta2), 2*np.pi - abs
                        (theta - theta2))
333                 if abs(rho - rho2) < min_rho_diff and circular_theta_diff <
                        min_theta_diff:
334                     duplicate = True
335                     break
336             if not duplicate:
337                 filtered_lines.append((rho, theta))
338         return np.stack(filtered_lines)
339
340
341
342     @staticmethod
343     def get_intersection_pts(horz_pts, vert_pts):
344
345         horz_lines = CornersDetector.get_homogenous_lines(horz_pts)
346         vert_lines = CornersDetector.get_homogenous_lines(vert_pts)
347
348         pts_hc = np.cross(horz_lines[:,None,:], vert_lines[None,:,:])
349         pts_hc = pts_hc.reshape(-1, pts_hc.shape[-1])
350         pts_hc /= pts_hc[:,-1][:,None]
351         return pts_hc[:,:-1]
352
353
354     @staticmethod
355     def get_homogenous_lines(pts):
356         """Given coordinates of pts, returns homogenous coordinates of lines
                """
357         pts_1 = np.concatenate([pts[:,:2], np.ones(pts.shape[0])[:,None]],
                axis=1)
358         pts_2 = np.concatenate([pts[:,2:], np.ones(pts.shape[0])[:,None]],
                axis=1)
359         lines = np.cross(pts_1, pts_2)
360         return lines
361
362     @staticmethod
363     def get_pts_on_lines(lines):
364         """Given coordinates of lines as rho, theta coordinates.
365         returns two points on the lines.
366         """
367         rhos = lines[:,0]
```

```
        thetas = lines[:,1]

        x_0s = rhos*np.cos(thetas)
        y_0s = rhos*np.sin(thetas)

        x1s = x_0s + 1000*(np.sin(-thetas))
        y1s = y_0s + 1000*(np.cos(thetas))
        x2s = x_0s - 1000*(np.sin(-thetas))
        y2s = y_0s - 1000*(np.cos(thetas))

        pts = np.stack([x1s, y1s, x2s, y2s], axis=-1)
        return pts


class HomographyEstimator:
    """Implements Homography estimation.
    """
    def __init__(self) -> None:
        """Creates estimator for homographic estimation.
        """
        pass

    @staticmethod
    def compute_pre_condtioning_transform(points_2d):

        centroid = np.mean(points_2d, axis=0)
        d = np.sqrt(np.sum((points_2d - centroid[None, :])**2, axis=1))
        d_avg = np.mean(d)

        # scale factor to set avg distance to sqrt(2)
        s = np.sqrt(2) / d_avg

        # pre-conditioning matrix
        T = np.eye(3)
        T[0,0] = s
        T[1,1] = s
        T[:2, -1] = -s*centroid
        return T


    @staticmethod
    def pre_condition_points(points_2d, T):
        """Isotropic scalling to pre-condition points."""

        points_hc = np.concat([points_2d, np.ones(points_2d.shape[0])[:,None
            ]], axis=1)
        points_norm = np.matmul(T, points_hc.T)
        return points_norm.T

    @staticmethod
    def get_coefficients_of_linear_equations(x, x_prime):
        """Return the coefficients of linear equations,
        for the given correspondence.

        Args:
            x = (3,)
            x_prime = (3,)
        """
```

```
425         return np.array([[0, 0, 0, -x_prime[2]*x[0], -x_prime[2]*x[1], -
                x_prime[2]*x[2], x_prime[1]*x[0], x_prime[1]*x[1], x_prime[1]*x
                [2]],
426                  [x_prime[2]*x[0], x_prime[2]*x[1], x_prime[2]*x[2], 0, 0, 0,
                      -x_prime[0]*x[0], -x_prime[0]*x[1], -x_prime[0]*x[2]]])
427
428     def get_coeffient_matrix(self, dom_pts, range_pts):
429         """For the given data points, get the coefficients matrix
430         A for homogeneous system of equations.
431
432         Args:
433             dom_pts: (N, 3)
434             range_pts: (N, 3)
435         """
436         A = []
437         for dom_pt, range_pt in zip(dom_pts, range_pts):
438
439             A_i = HomographyEstimator.get_coefficients_of_linear_equations(
                    dom_pt, range_pt)
440             A.append(A_i)
441         return np.concat(A, axis=0)
442
443     @staticmethod
444     def lst_square(A):
445         """Returns linear least square solution to homogeneous
446         system of linear equations, Ah = 0.
447         Since Rank(A) = n-1, solution is the eigen vector
448         corresponding to the smallest singular value of A
449         i.e. last row of vh.
450         """
451         u, s, vh = np.linalg.svd(A)
452         return vh[-1]
453
454
455     def get_homography(self, dom_pts, range_pts, precondition=False):
456         """Compute homography matrix
457         using linear least square fit.
458         """
459         if precondition:
460             T = HomographyEstimator.compute_pre_condtioning_transform(
                    dom_pts)
461             T_prime = HomographyEstimator.compute_pre_condtioning_transform(
                    range_pts)
462             dom_pts = HomographyEstimator.pre_condition_points(dom_pts, T)
463             range_pts = HomographyEstimator.pre_condition_points(range_pts,
                    T_prime)
464         else:
465             dom_pts = np.concat([dom_pts, np.ones(dom_pts.shape[0])[:, None
                    ]], axis=1)
466             range_pts = np.concat([range_pts, np.ones(range_pts.shape[0])[:,
                     None]], axis=1)
467
468         A = self.get_coeffient_matrix(dom_pts, range_pts)
469         h = HomographyEstimator.lst_square(A)
470
471         h /= h[-1]
472         H = h.reshape(3,3)
473         if precondition:
474             H_tilde = np.matmul(H, self.T)
```

```
            H = np.matmul(np.linalg.pinv(self.T_prime), H_tilde)
            H /= H[-1,-1]

        return H

    @staticmethod
    def apply_homography(H, physical_pts):
        """Apply homography matrix H, to physical points.
        Args:
            H= (3 x 3) homography matrix
            physical_pts= (N, 2) physical coordinates
        """
        hc_pts = np.concat([physical_pts, np.ones(physical_pts.shape[0])[:,
            None]], axis=1).transpose()
        hc_pts_prime = np.matmul(H, hc_pts)
        hc_pts_prime /= hc_pts_prime[-1]
        return hc_pts_prime[:2].transpose()


class Calibrator:
    def __init__(self, images, cell_length = 2.96, output_dir=None) -> None:
        """Camera calibration objects

        Args:
            images: list of open cv image objects
            cell_length: length of each cell on calibration pattern
        """
        # cell_length = 2.96 # cm or (1.1667 in)
        x_coord = np.arange(8)
        y_coord = np.arange(10)
        grid = np.meshgrid(x_coord, y_coord)
        self.physical_corners = cell_length*np.stack([grid[0].flatten(),
            grid[1].flatten()], axis=1)
        self.homograpy_estimator = HomographyEstimator()
        self.detector = CornersDetector(60, 100, 400, output_dir=output_dir)

        self.images = images
        self.num_images = len(images)
        self.homographies = []
        self.img_corner_pts = []
        for image in images:
            img_pts = self.detector.get_corner_pts(image)
            H = self.homograpy_estimator.get_homography(self.
                physical_corners, img_pts)
            self.homographies.append(H)
            self.img_corner_pts.append(img_pts)


    @staticmethod
    def get_coefficient_vector(h1, h2):
        """Return the coefficients of linear equations,
        for the given .

        Args:
            h1: column of homography matrix
            h2: column of homography matrix
        """
        return np.array([h1[0]*h2[0], h1[0]*h2[1]+h1[1]*h2[0], h1[1]*h2[1],
```

```
                    h1[2]*h2[0]+h1[0]*h2[2], h1[2]*h2[1]+h1[1]*h2[2], h1[2]*
                        h2[2]])

    @staticmethod
    def get_coefficient_for_Homography(H):
        """Return the matrix of coefficients for Vb=0 equation.

        Args:
            H: Homography matrix
        """
        h1 = H[:,0]
        h2 = H[:,1]
        v12 = Calibrator.get_coefficient_vector(h1,h2)
        v11 = Calibrator.get_coefficient_vector(h1,h1)
        v22 = Calibrator.get_coefficient_vector(h2,h2)
        V = np.array([v12,v11 - v22])
        return V

    def get_coeffient_matrix(self):
        """For the given data points, get the coefficients matrix
        A for homogeneous system of equations.

        Args:
            dom_pts: (N, 3)
            range_pts: (N, 3)
        """
        V = []
        for H in self.homographies:
            V_i = Calibrator.get_coefficient_for_Homography(H)
            V.append(V_i)

        return np.concat(V, axis=0)

    def calculate_intrinsic_parameters(self):
        """Using homographies from all available images, compute
        intrinsic parameters.
        """
        V = self.get_coeffient_matrix()
        b = HomographyEstimator.lst_square(V)

        y0 = (b[1]*b[3] - b[0]*b[4])/(b[0]*b[2] - b[1]*b[1])
        lmbda = b[5] - (b[3]*b[3] + y0*(b[1]*b[3] - b[0]*b[4]))/b[0]
        alpha_x = np.sqrt(lmbda/b[0])
        alpha_y = np.sqrt(lmbda*b[0]/(b[0]*b[2] - b[1]*b[1]))
        s = -b[1]*alpha_x*alpha_x*alpha_y/lmbda
        x0 = (s*y0/alpha_y) - (b[3]*alpha_x*alpha_x/lmbda)

        K = np.array([[alpha_x, s, x0],
                      [0, alpha_y, y0],
                      [0, 0, 1]])
        return K

    @staticmethod
    def calculate_extrinsic_parameters(K, H):
        """compute extrinsic parameters"""
        K_inv = np.linalg.pinv(K)
        r1 = K_inv@H[:,0]
        normalizer = np.linalg.norm(r1)
        r1 = r1/normalizer
```

```
        r2 = K_inv@H[:,1]/normalizer
        r3 = np.cross(r1, r2)
        t = K_inv@H[:,2]/normalizer

        R = np.stack([r1, r2, r3], axis=1)
        R = Calibrator.condition_orthonormal(R)
        return R, t

    @staticmethod
    def condition_orthonormal(R):
        """singular value condioning of R to make sure its
        columns are orthogonal."""
        u, s, vh = np.linalg.svd(R)
        R_cond = u@np.eye(s.size)@vh
        return R_cond

    @staticmethod
    def R_to_w(R):
        """Capture the 3 DoF of rotation matrix into 3-vector,
        using 'rodrigues_transform'
        """
        phi = np.arccos((np.trace(R)-1)/2)
        w = np.array([R[2,1] - R[1,2], R[0,2] - R[2,0], R[1,0] - R[0,1]])*
            phi/(2*np.sin(phi))
        return w

    @staticmethod
    def w_to_R(w):
        """Get rotation matrix R back from 'rodrigues_transform' w
        """
        phi = np.linalg.norm(w)
        w_cross = np.array([[0, -w[2], w[1]],
                            [w[2], 0, -w[0]],
                            [-w[1], w[0], 0]])
        R = np.eye(3) + (np.sin(phi)/phi)*w_cross + ((1-np.cos(phi))/phi**2)
            *(w_cross@w_cross)
        return R

    @staticmethod
    def get_free_param(K, R_matrices, t_vectors):
        """Capture the free parameters in a vector"""
        param = []
        param = [K[0,0], K[0,1], K[0,2], K[1,1], K[1,2]]
        for R,t in zip(R_matrices, t_vectors):
            w = Calibrator.R_to_w(R)
            param.extend([*w])
            param.extend([*t])

        return np.array(param)

    @staticmethod
    def get_calibration_parameters(param):
        """returns calibration parameters from free parameters."""
        R_matrices = []
        t_vectors = []

        K = np.eye(3)
        K[0,:] = param[:3]
        K[1,1:] = param[3:5]
```

```
        param = param[5:]
        num_param = param.size
        for i in range(num_param//6):
            p = param[6*i:6*(i+1)]
            w = p[:3]
            t = p[3:]
            R = Calibrator.w_to_R(w)
            R_matrices.append(R)
            t_vectors.append(t)
        return K, R_matrices, t_vectors

    @staticmethod
    def project_pts(K, R, t, pts):
        """Given calibration parameters and physical points,
        returns projected points.
        """
        pts_hc = np.concatenate(
            [pts, np.ones((pts.shape[0],1))],
            axis=1).transpose()

        H = K@np.stack([R[:,0],R[:,1], t], axis=1)

        proj_pts = H@pts_hc
        proj_pts /= proj_pts[-1]
        return proj_pts[:2].transpose()


    def compute_residuals(self, param):
        """Returns residuals for non-linear least squares."""
        residuals = []
        num_pts = self.physical_corners.shape[0]
        physical_corners_hc = np.concatenate(
            [self.physical_corners, np.ones((num_pts,1))],
            axis=1).transpose()
        K, R_matrices, t_vectors = Calibrator.get_calibration_parameters(
            param)
        for i in range(self.num_images):
            R = R_matrices[i]
            t = t_vectors[i]
            H = K@np.stack([R[:,0],R[:,1], t], axis=1)

            proj_pts = H@physical_corners_hc
            proj_pts /= proj_pts[-1]

            diff = self.img_corner_pts[i] - proj_pts[:2].transpose()
            residuals.extend(diff[:,0])
            residuals.extend(diff[:,1])

        return np.array(residuals)


    def compute_geometric_distance(self, param):
        """Returns geometric distance for the given parameters."""
        residuals = self.compute_residuals(param)
        return np.sum(residuals**2)

    def calibrate_camera(self):
```

```
        """Returns calibration parameters, both linear and refined using LM.
            """

        K = self.calculate_intrinsic_parameters()
        R_matrices = []
        t_vectors = []
        for H in self.homographies:
            R, t = self.calculate_extrinsic_parameters(K, H)
            R_matrices.append(R)
            t_vectors.append(t)

        print(f"Refining calibration parameters using non-linear least
            squares.")
        param = Calibrator.get_free_param(K, R_matrices, t_vectors)
        refined_param = scipy.optimize.least_squares(self.compute_residuals,
             param, method='lm').x
        K_ref, R_mats_ref, t_vec_ref = Calibrator.get_calibration_parameters
            (refined_param)

        return (K, R_matrices, t_vectors), (K_ref, R_mats_ref, t_vec_ref)

    def visualize_reprojections(self, index, calibration_param: tuple,
        lm_param=False):
        """Using the calibration parameters, visually compares detected and
        reprojected corner points.
        """
        if lm_param:
            identifier = f"using-LM-{index}"
        else:
            identifier = f"linear-{index}"

        img_pts = self.img_corner_pts[index]
        K, R_matrices, t_vectors = calibration_param
        proj_pts = Calibrator.project_pts(K, R_matrices[index], t_vectors[
            index], self.physical_corners)

        geo_error = np.sum((proj_pts - img_pts)**2)
        self.detector.plotter.visualize_projected_pts(
            image=self.images[index], pts=img_pts,proj_pts=proj_pts,
                geo_error=geo_error,
            identifier=identifier
            )



class Plotter:
    def __init__(self, output_dir=None) -> None:
        self.output_dir = output_dir

    def save_image(self, output_name, sub_dir=None):
        if sub_dir is None:
            filepath = os.path.join(self.output_dir, output_name)
        else:
            dir_path = os.path.join(self.output_dir, sub_dir)
            os.makedirs(dir_path, exist_ok=True)
            filepath = os.path.join(self.output_dir, sub_dir, output_name)

        plt.savefig(filepath)
        print(f"saved to path: {filepath}")
```

```
        plt.close()



    def display_img_with_lines(self, img, pts, ax=None, image_name=None,
        identifier=None, sub_dir=None):
        # Draw the lines
        if ax is None:
            fig, ax = plt.subplots()
        img = img.copy()
        pts = pts.astype(np.int32)
        for i in range(pts.shape[0]):
            pt = pts[i]
            cv2.line(img, (pt[0], pt[1]), (pt[2], pt[3]), (255,0,0), 2, cv2.
                LINE_AA)

        ax.imshow(img)
        if image_name is not None:
            if identifier is not None:
                title = f"{identifier}: {image_name[:-4]}"
                output_name = f'{identifier}-{image_name}'
            else:
                title = f"{image_name[:-4]}"
                output_name = f'lines-{image_name}'
            plt.title(title)
            self.save_image(output_name, sub_dir=sub_dir)


    def display_img_with_pts(
            self, img, pts, ax=None, image_name=None, fontsize=0.4,
            color=(255,0,0), label_pts=False, identifier=None, sub_dir=None,
            ):

        if ax is None:
            fig, ax = plt.subplots()
        # Draw the lines
        img = img.copy()
        pts = pts.astype(np.int32)
        for i in range(pts.shape[0]):
            x,y = pts[i]
            cv2.circle(img, (x, y), radius=3, color=color, thickness=-1)

            if label_pts:
                # Add the numbering next to the dot
                cv2.putText(img, f"{i+1}", (x + 5, y), fontFace=cv2.
                    FONT_HERSHEY_SIMPLEX,
                            fontScale=fontsize, color=color, thickness=1)

        ax.imshow(img)
        if image_name is not None:
            if identifier is not None:
                title = f"{identifier}: {image_name[:-4]}"
                output_name = f'{identifier}-{image_name}'
            else:
                title = f"{image_name[:-4]}"
                output_name = f'points_{image_name}'
            plt.title(title)

            self.save_image(output_name, sub_dir=sub_dir)
```

```
        return img



    def save_visualization(
            self, img, image_name=None, identifier=None, ax=None, sub_dir=
                None, ):
        if ax is None:
            fig, ax = plt.subplots()

        # Draw the lines
        ax.imshow(img, cmap='gray')
        if image_name is not None:
            if identifier is not None:
                title = f"{identifier}: {image_name[:-4]}"
                output_name = f'{identifier}-{image_name}'
            else:
                title = f"{image_name[:-4]}"
                output_name = f'{image_name}'
            plt.title(title)
            self.save_image(output_name, sub_dir=sub_dir)


    def visualize_projected_pts(self, image, pts, proj_pts, geo_error,
        identifier):
        """visualizing corners points and projected points,
        also adding error in the title.
        """
        sub_dir = 'reprojection'
        # Draw the lines
        img = 255*np.ones_like(image)
        pts = pts.astype(np.int32)
        pts_color = (255,0,0)
        proj_pts_color = (0,0,255)
        for i in range(pts.shape[0]):
            x,y = pts[i]
            cv2.circle(img, (x, y), radius=3, color=pts_color, thickness=-1)
            cv2.putText(img, f"{i+1}", (x - 17, y+10), fontFace=cv2.
                FONT_HERSHEY_SIMPLEX,
                            fontScale=0.4, color=pts_color, thickness=1)

            # projected points
            x,y = proj_pts[i].astype(np.uint16)
            cv2.circle(img, (x, y), radius=3, color=proj_pts_color,
                thickness=-1)
            cv2.putText(img, f"{i+1}", (x + 2, y+10), fontFace=cv2.
                FONT_HERSHEY_SIMPLEX,
                            fontScale=0.4, color=proj_pts_color, thickness
                                =1)

        title = f"{identifier}, geometric error: {geo_error:.2f}"
        output_name = f'reprojection-{identifier}.jpg'
        plt.imshow(img)
        plt.title(title)
        self.save_image(output_name, sub_dir=sub_dir)


    @staticmethod
    def world_to_cam_coord(X, R, t):
```

```
        X_cam = R@X + t
        return X_cam

    @staticmethod
    def cam_to_world_coord(X_cam, R, t):

        C = -R.T@t
        X = R.T@X_cam + C
        return X

    @staticmethod
    def get_line_in_3D(origin, head, color='r', width=5, name='X'):
        origin = origin
        head = head
        # Define the line from start to end for each vector
        vector_line = go.Scatter3d(
            x=[origin[0], head[0]],
            y=[origin[1], head[1]],
            z=[origin[2], head[2]],
            mode='lines',
            line=dict(color=color, width=width),  # Set color and width of
                the vector line
            name=name,
        )

        # adding arrow to line
        cone_arrows = go.Cone(
            x=[head[0]],
            y=[head[1]],
            z=[head[2]],
            u=[head[0]-origin[0]],
            v=[head[1]-origin[1]],
            w=[head[2]-origin[2]],
            colorscale=[[0, color], [1, color]],
            sizemode="absolute",
            showscale=False,
            sizeref=5,  # Adjust size as needed
        )
        # cone_arrows=None

        return vector_line, cone_arrows

    @staticmethod
    def random_color_with_alpha(alpha=0.5):
        return f'rgba({np.random.randint(0, 255)}, {np.random.randint(0,
            255)}, {np.random.randint(0, 255)}, {alpha})'

    @staticmethod
    def create_camera_principle_plane(R, t, alpha=0.6, size=15, index=0):
        """Create a camera plane for the given pos"""
        x = np.linspace(-size, size, 2*size)
        y = np.linspace(-size, size, 2*size)
        X, Y = np.meshgrid(x, y)
        Z = np.zeros_like(X)

        X = X - t[0]
        Y = Y - t[1]
        Z = Z - t[2]

```

```
        # Apply the rotation to the X, Y coordinates
        coordinates = np.vstack([X.flatten(), Y.flatten(), Z.flatten()])
        rotated_coordinates = np.dot(R.T, coordinates)

        # # Reshape the rotated coordinates back to 2D
        X = rotated_coordinates[0].reshape(X.shape)
        Y = rotated_coordinates[1].reshape(Y.shape)
        Z = rotated_coordinates[2].reshape(Z.shape)

        surface = go.Surface(
            z=Z,
            x=X,
            y=Y,
            surfacecolor=np.full_like(X, index),  # You can set surfacecolor
                based on Z values for variation
            colorscale=[[0, Plotter.random_color_with_alpha(alpha)], [1,
               Plotter.random_color_with_alpha(alpha)]],
            showscale=False  # Hide the color scale bar
        )

        C = -R.T@t
        # print(f"Center: {C}")
        X_cam_x = np.array([size, 0, 0])
        X_x = Plotter.cam_to_world_coord(X_cam_x, R, t)
        line_x, cone_x = Plotter.get_line_in_3D(C, X_x, 'red', name='X')

        # y line
        X_cam_y = np.array([0, size, 0])
        X_y = Plotter.cam_to_world_coord(X_cam_y, R, t)
        line_y, cone_y = Plotter.get_line_in_3D(C, X_y, 'green', name='Y')

        # z line
        X_cam_z = np.array([0,0, size])
        X_z = Plotter.cam_to_world_coord(X_cam_z, R, t)
        line_z, cone_z = Plotter.get_line_in_3D(C, X_z, 'blue', name='Z')

        return surface, [line_x, line_y, line_z], [cone_x, cone_y, cone_z]

    @staticmethod
    def create_calibration_pattern(size_scaler=3, alpha=0.5):
        s = size_scaler
        x = np.linspace(0, 7*s, 100)  # 10 points along x-axis from 1 to 2
        y = np.linspace(0, 9*s, 100)  # 10 points along y-axis from 1 to 2
        X, Y = np.meshgrid(x, y)   # Create a mesh grid for X and Y

        mask_x = (X>0)&(X<1*s)|(X>2*s)&(X<3*s)|(X>4*s)&(X<5*s)|(X>6*s)&(X<7*
            s)
        mask_y = (Y>0)&(Y<1*s)|(Y>2*s)&(Y<3*s)|(Y>4*s)&(Y<5*s)|(Y>6*s)&(Y<7*
            s)|(Y>8*s)&(Y<9*s)
        mask = mask_x & mask_y
        # Z values for the surface (constant value, making it flat)
        Z = np.zeros_like(X)
        C = np.zeros_like(X)
        C[mask] = 1

        # Create the surface plots
        surface = go.Surface(
            z=Z,
            x=X,
```

```
            y=Y,
            surfacecolor=C,
            colorscale=[[0, 'white'], [1, 'black']],
            showscale=False
        )
        return surface



    def make_3D_plot(self, R_matrices, t_vectors, dataset_name, alpha=0.5, size=20, area_size=80,calib_pattern_size=5):
        all_surfaces = []
        all_lines = []
        all_cones = []
        for index, (R, t) in enumerate(zip(R_matrices, t_vectors)):
            surface, lines, cones = Plotter.create_camera_principle_plane(R, t, alpha=alpha, size=size, index=index)
            all_lines.extend(lines)
            all_cones.extend(cones)
            all_surfaces.append(surface)

        all_surfaces.append(Plotter.create_calibration_pattern(calib_pattern_size, alpha=alpha))
        fig = go.Figure(data=all_surfaces+all_lines+all_cones)

        # Set the layout options
        fig.update_layout(
            showlegend=False,
            scene=dict(
                xaxis_title='X',
                yaxis_title='Y',
                zaxis_title='Z',
                xaxis=dict(range=[-60, area_size]),
                yaxis=dict(range=[-20, area_size]),
                zaxis=dict(range=[-area_size, 5]),  # Adjust Z range to include all surfaces
                camera=dict(
                    eye=dict(x=1, y=-1.25, z=-1),  # Position of the camera (e.g., diagonally above)
                    center=dict(x=0.5, y=0, z=0),          # Center point the camera looks at (scene center)
                    up=dict(x=0, y=-1, z=0)               # Defines the "up" direction of the camera
                )
            ),
            title=f'Camera poses in 3D ({dataset_name})'
        )
        fig.show()
        # image_path = os.path.join(self.output_dir, f'3D-view-camera-poses-{dataset_name}.png')
        # fig.write_image(image_path, width=600, height=400, scale=1)
        # print(f"3D view of poses saved to: {image_path}")


if __name__ == '__main__':
    calibration_datasets = [
        r'C:\Users\ahmedb\projects\computer-vision/Auxilliary/calibration_dataset1',
```

```
        r'C:\Users\ahmedb\projects\computer-vision/Auxilliary/calibration_dataset2'
    ]

    output_dirs = [
        r'C:\Users\ahmedb\projects\computer-vision/images/hw8/dataset1',
        r'C:\Users\ahmedb\projects\computer-vision/images/hw8/dataset2',
    ]
    fixed_images = ['Pic_13.jpg', 'Pic_1.jpg']

    for i, (calibration_dataset, output_dir) in enumerate(zip(calibration_datasets, output_dirs)):

        image_filepaths = glob.glob(os.path.join(calibration_dataset, '*.jpg'))


        # visualize corner detection steps
        image_files = [os.path.basename(file) for file in image_filepaths]
        detector = CornersDetector(60, 100, 400, output_dir=output_dir)
        for _ in range(5):
            image_name = np.random.choice(image_files)
            image_path = os.path.join(calibration_dataset, image_name)
            print(f"Reading image: {image_name}...", end='')
            image = read_image_from_path(image_path)

            detector.visualize_Hough_lines_extraction(image, image_name=image_name, subdir='corner_detection')


        # Calibration using linear and LM
        images = [read_image_from_path(image_path) for image_path in image_filepaths]
        calibrator = Calibrator(images, cell_length=2.96, output_dir=output_dir)

        linear_param, lm_param = calibrator.calibrate_camera()
        print(f"K: {linear_param[0]}")
        print(f"K (LM): {lm_param[0]}")
        for _ in range(5):
            index = np.random.randint(len(images))
            calibrator.visualize_reprojections(index=index, calibration_param=linear_param, lm_param=False)
            print(f"index: {index}")
            print(f"R: {linear_param[1][index]}")
            print(f"t: {linear_param[2][index]}")
            calibrator.visualize_reprojections(index=index, calibration_param=lm_param, lm_param=True)
            print(f"R: {lm_param[1][index]}")
            print(f"t: {lm_param[2][index]}")

        index = 0
        print(f"Fixed pose")
        calibrator.visualize_reprojections(index=index, calibration_param=linear_param, lm_param=False)
        print(f"R: {linear_param[1][index]}")
        print(f"t: {linear_param[2][index]}")
        calibrator.visualize_reprojections(index=index, calibration_param=lm_param, lm_param=True)
```

```
        print(f"R: {lm_param[1][index]}")
        print(f"t: {lm_param[2][index]}")
        print(f"Camera center = {-lm_param[1][index].T@lm_param[2][index]}")

        # visualizing 3D camera view
        K, R_matrices, t_vectors = lm_param
        calibrator.detector.plotter.make_3D_plot(
            R_matrices, t_vectors, 'dataset1', alpha=0.4, size=15,
                calib_pattern_size=5)
```

Listing 1: Example Python code